

Hotel Price Calculator — How It Works

Data model & Directus realization · covers Room Prices and Surcharges

This document explains, in plain language, how a hotel's **buy price** (what we pay the hotel) turns into the **sell price** (what the customer pays) for both room prices and surcharges — and exactly where that logic lives inside Directus. It's written so it can be dropped into Claude's project context and used as a lasting reference.

1. The Big Picture

For every hotel, two things exist in parallel, for every sellable market (Germany, Switzerland, the Netherlands, etc.):

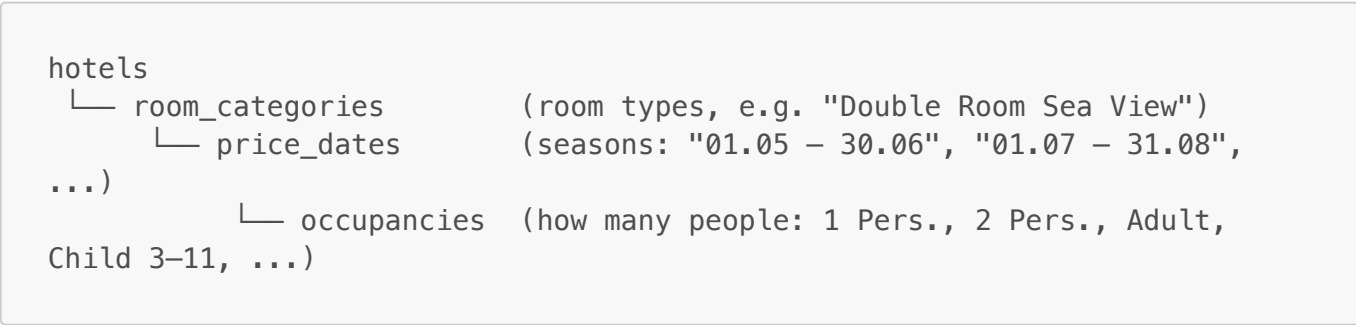
- 1. **A buy price** — one cost figure per room type / per surcharge, entered once by an editor.
- 2. **A sell price per market** — automatically calculated from the buy price using that market's **margin %** and **exchange rate**.

Because margin and exchange rate can differ per market, the *same* buy price can legitimately produce different sell prices for German customers vs. Swiss customers vs. Dutch customers. That's the core idea the whole system is built around.

The calculation itself is not done by a human, and it's not a spreadsheet formula — it's a small piece of JavaScript inside a **Directus Flow**, triggered by a button in the hotel's editing screen. Editors only ever touch buy prices, margins, and exchange rates; the sell price is always machine-calculated and read-only.

2. Part A — The Data Model

2.1 The core chain



Every combination of **room category** × **price date** × **occupancy** for a given hotel gets its own row in **room_prices** — this is the actual price grid an editor fills in.

2.2 The price matrix (the heart of the system)

Collection	What it holds	Key fields
------------	---------------	------------

Collection	What it holds	Key fields
<code>room_prices</code>	One row per (room category × price date × occupancy) for a hotel. This is a junction table — it only exists because three different things (category, date, occupancy) need to combine.	<code>hotel_id</code> , <code>room_category_id</code> , <code>price_date_id</code> , <code>room_occupancy_id</code> , <code>buy_price</code>
<code>room_prices_translations</code>	One row per <code>room_prices</code> row per market/language . This is where the calculated sell price actually lives.	<code>room_prices_id</code> , <code>translations_id</code> (the market), <code>sell_price</code> (read-only, machine-written)

So a single room type, in a single season, for "2 persons," has **one** `room_prices` row (the buy price), and then **one** `room_prices_translations` row **per market** (de-DE, de-CH, nl-NL...), each with its own calculated sell price.

2.3 Per-market pricing rules

`hotels_translations_1` is a junction between `hotels` and `translations` (the market list) that carries the **settings** used to turn a buy price into a sell price, one row per hotel per market:

Field	Meaning
<code>margin_percentage</code>	The markup % applied for this market
<code>exchange_rate</code>	Which currency-conversion rate (from the <code>rates</code> collection) applies
<code>percentage_type</code>	net or gross — changes the formula shape (see Part B)
<code>provision_percentage</code>	Only used when <code>percentage_type</code> = gross
<code>buy_price_type</code> / <code>sell_price_type</code>	per_unit (whole room/apartment) or per_person — if these differ, the sell price gets divided by the occupancy's headcount
<code>from_price</code>	Points at the cheapest <code>room_prices</code> row — this is the "from €X" teaser price shown on the hotel

2.4 Surcharges

Surcharges (city tax, breakfast, pet fee, resort fee, etc.) follow the exact same buy-price → per-market-sell-price pattern, just without the room/date/occupancy matrix:

Collection	What it holds
<code>surcharges</code>	One row per surcharge for a hotel: <code>name</code> , <code>buy_price</code> , publish dates

Collection	What it holds
<code>surcharges_translations</code>	One row per surcharge per market : <code>sell_price</code> (calculated), plus the translated <code>surcharge_booking_name/surcharge_description</code> , and two lookup links: <code>surcharge_type</code> → is it mandatory or optional? and <code>surcharge_calc_type</code> → how is it charged?
<code>hotels_surcharges_translations</code>	Per hotel, per market: <code>surcharge_margin_percentage</code> , <code>surcharge_exchange_rate</code> , <code>surcharge_percentage_type</code> , <code>surcharge_provision_percentage</code> — mirrors <code>hotels_translations_1</code> but scoped to surcharges

Two small lookup tables give the actual business meaning behind a surcharge:

- **mandatory** — *optional* or *obligatorisch* (mandatory). Whether the guest can decline it.
- **calculation_method** — *einmalig pro Einheit* (once per unit), *einmalig pro Person* (once per person), *pro Einheit und Nacht* (per unit per night), *pro Person und Nacht* (per person per night). How the charge scales with stay length and party size.

2.5 Supporting reference data

Collection	Purpose
<code>rates</code>	Currency exchange rates, each with a validity window (<code>valid_from</code> / <code>valid_to</code>) and a from-currency → to-currency pair
<code>currencies</code>	The currency list (code, symbol)
<code>translations</code>	The list of markets/languages a hotel can be sold in (e.g. de-DE, de-CH, nl-NL). German (de-DE) is treated as the "master" market — see Part B
<code>occupancies</code>	Global lookup of party sizes/ages (Adult, Child 3–11, 2 Pers., etc.), each with a <code>value</code> (headcount) and a <code>from_price</code> flag used for the teaser price
<code>valid_on_weekdays</code>	Mo–So — lets a room category be split into weekday-specific pricing variants

2.6 Global defaults that seed new hotels

Rather than every new hotel starting with blank margin/exchange-rate fields, two global preset collections exist:

- **margin_presets** — one default margin % per market, with optional **per-destination overrides** in `mp_hotel_destination_exceptions` (e.g. "Greece uses a different margin than the default").
- **exchange_rate_presets** — same idea for exchange rates, with overrides in `erp_hotel_destination_exceptions`.

When a hotel is created, these presets automatically fill in its `hotels_translations_1` / `hotels_surcharges_translations` rows — but only where a value is still empty, so they never

overwrite something an editor already typed in.

Note — a separate, simpler tool exists too: the `batch_hotel` / `batch_hotel_translations` collections power a *different*, flatter bulk-import calculator ($\text{sell} = \text{buy} \times \text{exchange_rate} \times (1 + \text{margin}/100)$), no room/date/occupancy matrix). It's used for bulk-imported staging data, not the main hotel content type described here — worth knowing so the two aren't confused if you come across it.

3. Part B — How It's Built in Directus

3.1 The editing screen (custom extensions)

Editors never touch `room_prices` or `surcharges_translations` directly in a raw table — two custom Directus **interface extensions** provide a proper spreadsheet-like experience:

- **directus-extension-interface-room-prices-table** — shown inside a hotel record. Renders room categories as rows, grouped by season (price date) and occupancy, with a market/language switcher. Editors type in **buy prices only**; sell prices are shown read-only. A **"Save & Calculate"** button triggers the real calculation (below).
- **directus-extension-interface-surge-price** — same pattern, simpler layout, for surcharges.
- **directus-extension-interface-occupancy-selector** — lets an editor pick which occupancy bands (Single, Double, Triple, Child bands...) a given hotel actually offers.

3.2 Keeping the price grid populated (housekeeping flow)

Whenever something structural changes — a new room category, a new season, a new occupancy added to a hotel — the grid of `room_prices` rows needs to grow to match. This is handled by the **Sync Hotel Room Prices** flow:

- Reads all room categories, price dates, and occupancies for the hotel.
- Cross-joins them (room category × price date × occupancy) and creates any **missing room_prices** rows, with `buy_price` defaulting to 0 until an editor fills it in.
- Also creates any missing `room_prices_translations` rows (one per market) so there's always something for the calculator to write into.
- It's re-triggered automatically on: hotel create/update, new room category, new season, new occupancy, or a new market being added — so the grid is always kept in sync without manual intervention.

This flow **does not calculate any money** — it only makes sure the right empty cells exist.

3.3 The actual calculation — Room Prices

This is the part that answers "how is the price calculated." It's a webhook-triggered flow called **Calculate Room Prices**, fired by the "Save & Calculate" button. For a given hotel, it reads every market's pricing settings (`hotels_translations_1`) and every `room_prices` buy price, then for each one:

```
converted_buy = buy_price × exchange_rate

if percentage_type == "net":
    sell = converted_buy / (1 - margin)

if percentage_type == "gross":
    sell = converted_buy × (1 - provision) / (1 - margin)

# If the hotel is bought per unit/apartment but sold per person (or vice versa),
# divide by the number of people in that occupancy:
if buy_price_type != sell_price_type:
    sell = sell / occupancy_headcount

# Round to the nearest whole currency unit (e.g. 19.50 → 20, 19.49 → 19)
sell = round(sell)
```

The result is written into `room_prices_translations.sell_price`. While this runs, the hotel record's `sell_prices_status` field flips `idle` → `processing` → `done` (or `failed`), which is what powers the little status indicator editors see, along with a `sell_prices_updated_at` timestamp.

3.4 The actual calculation — Surcharges

A near-identical webhook flow, **Calculate Surcharge Prices**, does the same job for surcharges — reading `hotels_surcharges_translations` for the margin/exchange settings and `surcharges.buy_price` for the cost, applying the exact same net/gross formula, for every surcharge × every market:

```
converted_buy = buy_price × exchange_rate

if surcharge_percentage_type == "net":
    sell = converted_buy / (1 - margin)

if surcharge_percentage_type == "gross":
    sell = converted_buy × (1 - provision) / (1 - margin)

sell = round(sell)
```

The one difference from room prices: **surcharges are never divided by occupancy** — a city tax or resort fee doesn't scale by headcount inside this calculation (that scaling, if any, is applied later at booking time based on `calculation_method`, not here). The result is written into `surcharges_translations.sell_price`.

3.5 Where margin % and exchange rate actually come from

Three mechanisms feed the settings that the formulas above depend on:

1. **Global presets on creation** — when a hotel is created, a flow checks `margin_presets` / `exchange_rate_presets` (with destination-specific overrides checked first) and fills in any still-empty per-market fields.
2. **German (de-DE) as the master market** — a housekeeping flow copies `buy_price_type`, `sell_price_type`, `percentage_type`, `exchange_rate`, and `provision_percentage` from the de-DE row to any other market row that's still empty. In practice, editors set things up once in German and other markets inherit it by default.
3. **Manual "copy settings" button** — the **Save Settings For All Languages** action lets an editor explicitly pick a source market and push its room-price and/or surcharge settings to every other market, either only filling blanks or fully overriding everything.

3.6 The "from €X" teaser price

Hotel listings show a headline "from €X" price. This is computed by the **From Price Selection** flow: after prices are recalculated, it looks at every room price flagged as a from-price candidate (via the room category's `price_start` flag, the season's `from_price` flag, or the occupancy's `from_price` flag), picks the **cheapest sell price per market**, and stores a pointer to it on `hotels_translations_1.from_price`. If nothing is explicitly flagged, it falls back to the overall cheapest room price.

3.7 Getting the calculated prices out to the public website

A custom REST API extension (`api`) exposes `GET /api/v1/hotels/:id?lang=xx`. This is the endpoint the public booking website actually calls — it returns the hotel's rooms and surcharges already shaped for that specific market, with buy price, calculated sell price, and margin included, and automatically filters out anything unpublished or outside its active date range.

4. Worked Example

Say a hotel's "Double Room, Sea View" costs **€100/night** (buy price) for 2 persons in high season, and the German market (de-DE) is set up as:

- Margin: **20%**
- Exchange rate: **1.00** (same currency)
- Type: **net**, buy and sell both **per unit**

```
converted_buy = 100 × 1.00 = 100  
sell = 100 / (1 - 0.20) = 100 / 0.80 = 125
```

→ The customer sees **€125/night**.

Now say the Swiss market (de-CH) sells in CHF with an exchange rate of **1.05** and a margin of **25%**:

```
converted_buy = 100 × 1.05 = 105  
sell = 105 / (1 - 0.25) = 105 / 0.75 = 140
```

→ The Swiss customer sees **CHF 140/night** — same room, same buy price, different market settings.

5. Quick Reference

Key Flows

Flow	Trigger	What it does
Calculate Room Prices	Webhook (Save & Calculate button)	Runs the buy→sell formula for every room price × market on a hotel
Calculate Surcharge Prices	Webhook (Save & Calculate button)	Same formula for every surcharge × market
Sync Hotel Room Prices	Auto, on hotel/category/season/occupancy changes	Keeps the <code>room_prices</code> grid complete (structure only, no money math)
[Margin Preset] Hotel create/update	Auto, on hotel create/update	Seeds empty margin fields from global presets / destination exceptions
[Hotel Prices] Sync translations from de-DE	Auto, on settings change	Cascades settings from the German master row to empty markets
Save Settings For All Languages	Manual button	Lets an editor explicitly copy settings to all markets
From Price Selection	Auto, after price recalculation	Computes the "from €X" teaser price per market

Key Collections

Collection	Role
<code>room_prices</code>	Buy price per room category × season × occupancy (junction table)
<code>room_prices_translations</code>	Calculated sell price per market
<code>hotels_translations_1</code>	Per-market margin / exchange-rate / pricing-type settings
<code>surcharges / surcharges_translations</code>	Same buy → sell pattern for surcharges
<code>hotels_surcharges_translations</code>	Per-market margin / exchange-rate settings for surcharges
<code>rates</code>	Currency exchange rates
<code>margin_presets / exchange_rate_presets</code>	Global defaults that seed new hotels

Collection	Role
<code>mandatory / calculation_method</code>	Surcharge business rules (mandatory/optional, how it's charged)

Key Extensions

Extension	Role
<code>directus-extension-interface-room-prices-table</code>	The room-price editing grid
<code>directus-extension-interface-surcharge-prices</code>	The surcharge editing grid
<code>directus-extension-interface-occupancy-selector</code>	Picking which occupancy bands a hotel offers
<code>api</code>	Public REST API that serves the calculated prices to the booking website